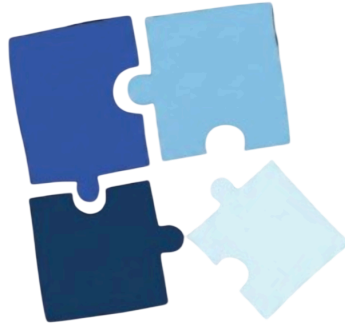




CoMatch

Convenient Teammate Matching for All Purposes

Project for Orbital 2026

[GitHub Repository](#)

Authors:

Win Htut Khaung Soe and Nay Min Thar

Proposed Level of Achievement:

Project Gemini

Advisor:

Eugene Oh Yun Zheng

Table of Contents

Table of Contents.....	2
Scope.....	3
Brief Description.....	3
Motivation.....	3
Our Solution.....	3
Competitive Analysis.....	3
Target Audience.....	4
Core Features.....	5
01: User Authentication.....	5
02: User Profile Building.....	7
03: Spaces.....	9
04: Post.....	12
05: Dashboard.....	15
06: Real-time 1-on-1 Chat.....	17
Tech Stacks.....	18
Software Engineering Principles.....	18
Separation of Concerns (SoC).....	18
Technical Proof of Concept.....	19
Integration of Essential Components.....	19
Implemented Features & System Integration.....	19
Database Schema.....	21
Project Management.....	21
Instruction for Testing.....	21
Development Plan.....	22
Project Log.....	23

Scope

Brief Description

CoMatch is a centralized, intuitive platform designed to help students and young adults easily connect with compatible teammates for hackathons, academic projects, and coding competitions.

Motivation

Students and young adults frequently demonstrate a strong enthusiasm for self-improvement through participation in hackathons, academic projects, and competitions. However, the prerequisite of forming a team often poses a significant barrier. It is notoriously difficult to secure compatible teammates who share equivalent motivation, complementary skills, and aligned objectives. This friction is evident even within university programs like Orbital, where a substantial number of potential participants are unable to join simply due to the inability to find a like-minded partner.

Our Solution

The primary objective of this project is to build a centralized, intuitive platform that facilitates the connection between students and suitable teammates for specific competitions or projects. The system will enable users to create dedicated project spaces, publish teammate recruitment posts, and browse existing opportunities. By streamlining the discovery and communication phases, we aim to optimize the team formation process and empower more students to participate in collaborative initiatives.

Competitive Analysis

1. Ad-Hoc Messaging & Spreadsheets (Telegram, MS Teams, Google Sheets)

This is the current de facto method for finding teammates for most programmes and university modules that involve teamwork. Coordinators often share a massive Google Sheet, or students resort to posting text blurbs in huge Telegram/ MS Teams groups.

- **The Issue:** It is highly disorganized and ephemeral. In a chat group with hundreds of students, recruitment messages get buried within minutes. On spreadsheets, data goes stale quickly, and there is no way to know if a person has already found a team without messaging them directly. Furthermore, rejecting a teammate via a direct message carries social friction, leading to "ghosting."
- **CoMatch's Solution:** CoMatch replaces chaos with structure. Instead of unstructured chat text, CoMatch uses standardized User Profiles linked directly to LinkedIn for instant skill verification. More importantly, CoMatch's Applicant Workflow and Dashboard completely removes the social friction of team building. Users can cleanly manage, accept, or reject join requests from a central dashboard, keeping the process professional and organized.

2. Hackathon-Specific Platforms (Devpost / HackerEarth)

Devpost is the industry giant for hackathons, and they feature a built-in "Find Teammates" tab for users participating in their events.

- **The Issue:** These platforms are strictly "walled gardens." The team-finding feature is entirely locked to specific events hosted by official organizers on their platform. If a student wants to find a partner for an independent summer project, a startup idea, or a university module (like Orbital) that isn't hosted on Devpost, the platform is entirely useless to them.
- **Why CoMatch is Better:** CoMatch is event-agnostic and user-driven. Through our "Spaces" feature, any user can take the initiative to create a hub for a project, whether it's an official university module, a niche local competition, or just a weekend side-project. CoMatch empowers the students to create the environment, rather than waiting for an official event organizer to do it for them.

3. Institutional Team-Makers (e.g., CATME, Canvas LMS Groups)

Tools like CATME are frequently used by university professors to group students together for academic projects based on surveys (schedules, GPA, basic skills).

- **The Issue:** These solutions are strictly top-down, algorithmic, and forced. They completely remove student agency. Algorithms group students based on schedule matrices but entirely ignore personal dynamics, communication styles, and mutual passion for a specific idea. Furthermore, these platforms are notoriously outdated, with clunky UI/UX that students generally dread using.
- **Why CoMatch is Better:** CoMatch is peer-to-peer and completely voluntary. It treats students like young professionals networking for a startup, rather than children being assigned a group project. By allowing users to browse specific project ideas, leave public comments for clarification, and talk to each other directly before sending or accepting join requests, CoMatch ensures that teams are formed based on mutual compatibility and shared vision.

Target Audience

CoMatch is designed to serve a highly motivated demographic of students, young professionals, and creators who actively seek collaborative opportunities. The following are three main groups of target users.

1. Students and academic cohorts (teamwork in academic modules, team-based projects, etc.)
2. Competition participants (hackathons, business plan competitions, etc.)
3. Independent creators (startups, open-source projects, etc.)

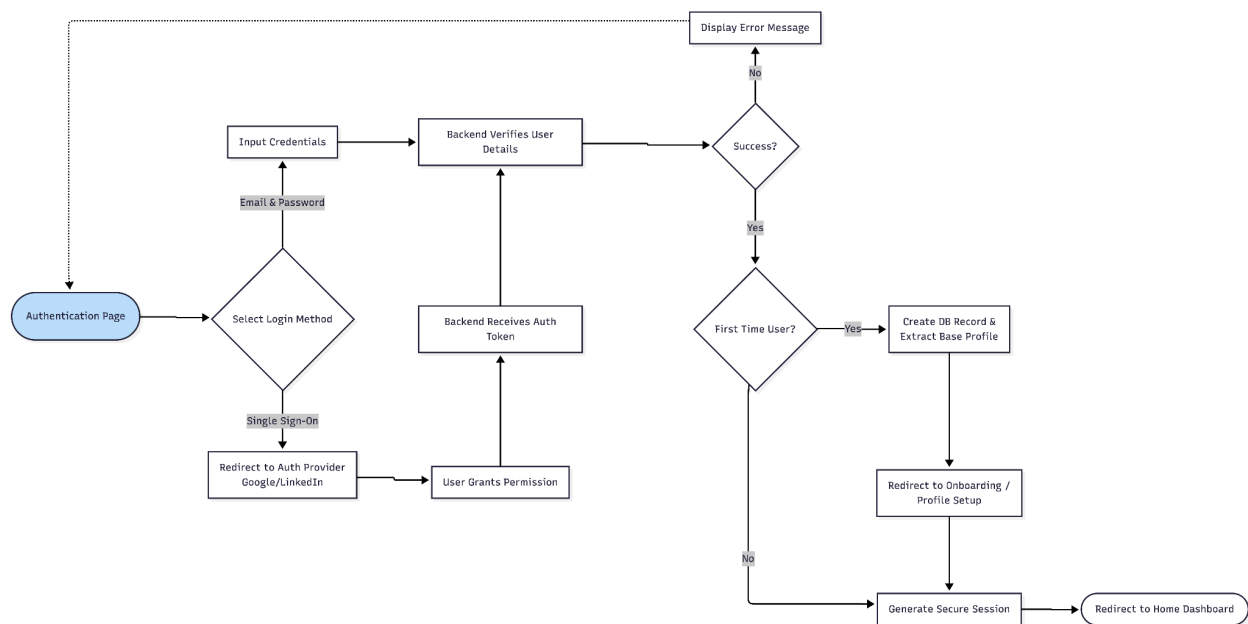
Core Features

01: User Authentication

Overview

User Authentication serves as the platform's core security layer. It ensures that all project applications, user profiles, and private chats are tied to verified, accountable users, establishing the trust necessary for reliable team matching. We will provide three methods of login/ signup: Email and Password, Google and LinkedIn. On successful login, users will be redirected to the Home Page.

Activity Diagram



Mockups

 **CoMatch**

Welcome!
Connect . Collaborate . Conquer

Email

Password

☐ Remember me [Forgot Password?](#)

Sign in

Or Continue With

 Continue with Google

 Continue with LinkedIn

Don't have an account? [Sing up here!](#)

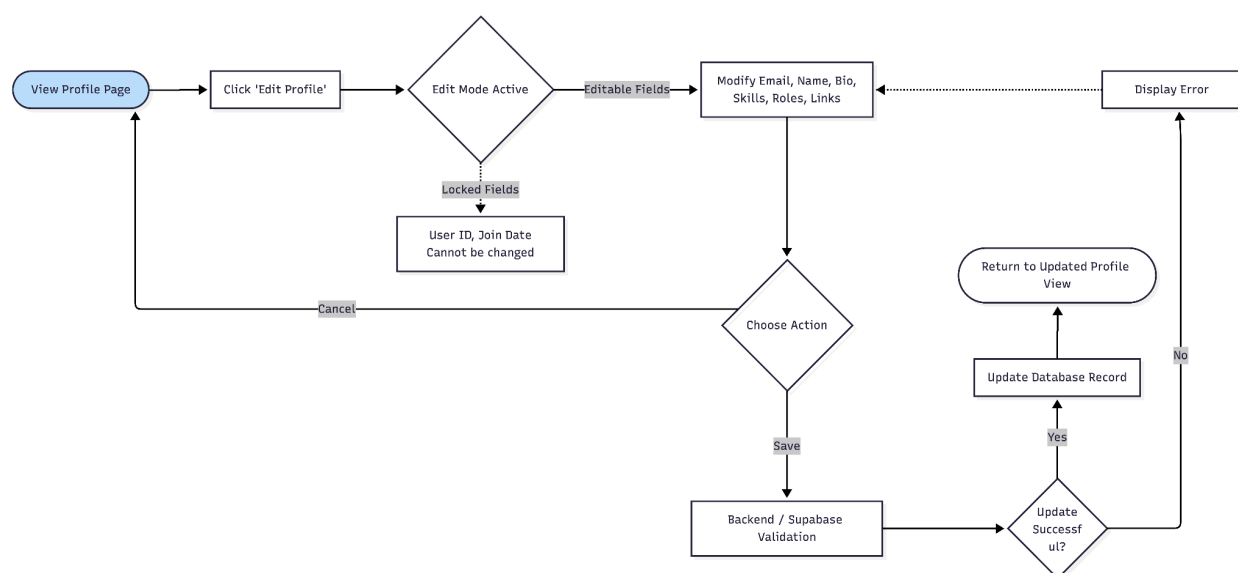
02: User Profile Building

Overview

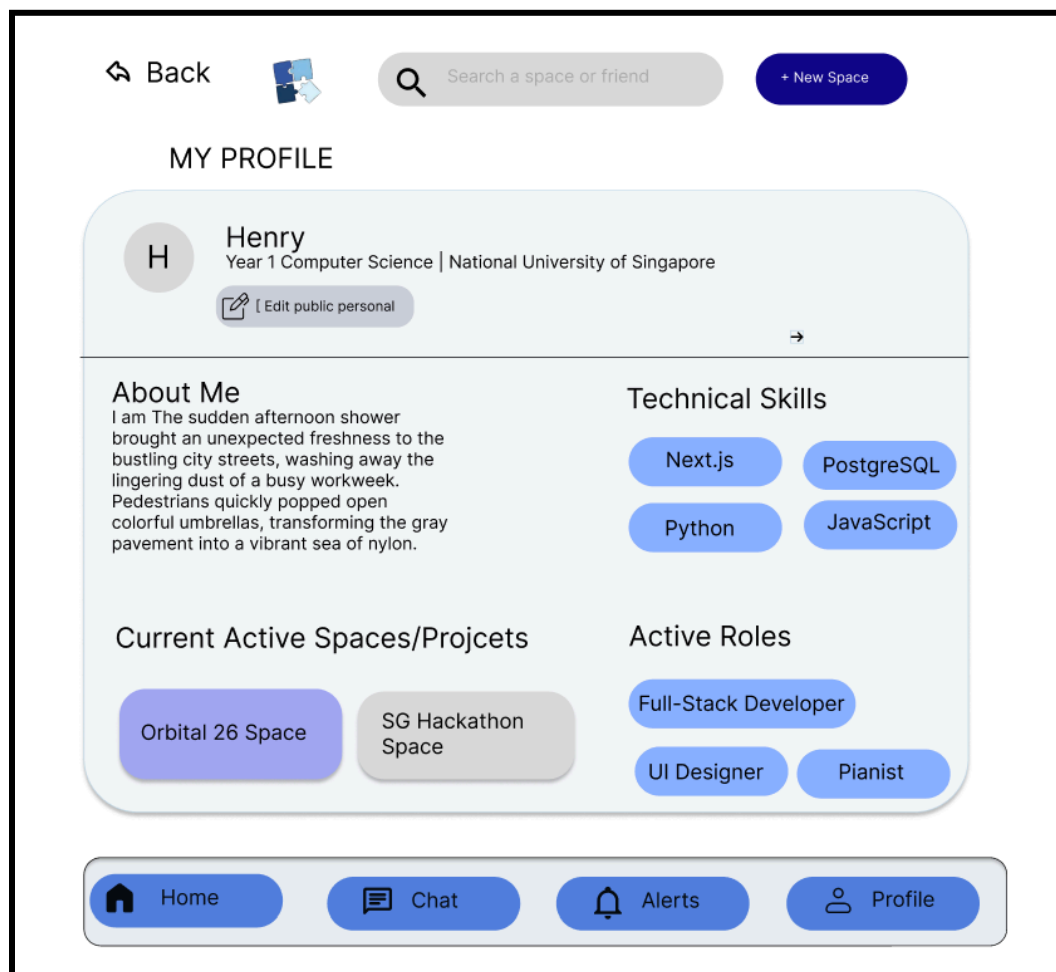
To foster trust and facilitate effective team matching, a comprehensive User Profile system is required. A user's profile acts as their digital resume on the platform, showcasing their technical skills, current roles, and professional background to potential teammates and project creators.

While a user's core identity metrics (id, created_at) are locked upon registration to maintain system stability, users retain full control over their public-facing persona such as name, profile picture, etc. Authenticated users can navigate to their settings to edit their profiles at any time.

Activity Diagram



Mockups



03: Spaces

Overview

A "Space" acts as the foundational hub within the Assembly platform. It represents a specific academic module, hackathon, competition, or independent project category. Before any user can seek teammates or post recruitment listings, a dedicated Space for that specific initiative must exist. This feature enables a community-driven approach to organizing project opportunities through two primary functions: Space Creation and Space Editing.

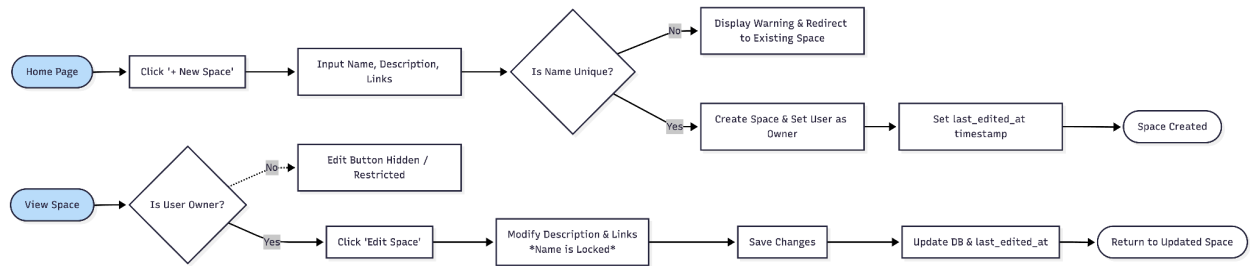
1. Space Creation Users can browse existing Spaces on the home page. If a specific project or module is not yet listed, any authenticated user can create a new Space using the "Add" function.

- **Properties:** Upon creation, the user must input the Space's Name, a Description, and relevant External Links (e.g., to a Devpost page or official module outline).
- **Uniqueness Constraint:** To prevent fragmentation and ensure users can effectively find opportunities, the "Name" of a Space must be globally unique. Upon form submission, the system verifies the proposed name against existing records. If a duplicate is detected, the system blocks the creation and displays a warning prompt redirecting the user to the existing Space.
- **Immutability:** Once successfully created, a Space cannot be deleted. This strict non-deletion policy is crucial for system stability; deleting a Space could orphan active recruitment posts and active teams currently operating within it, leading to data corruption and a poor user experience.

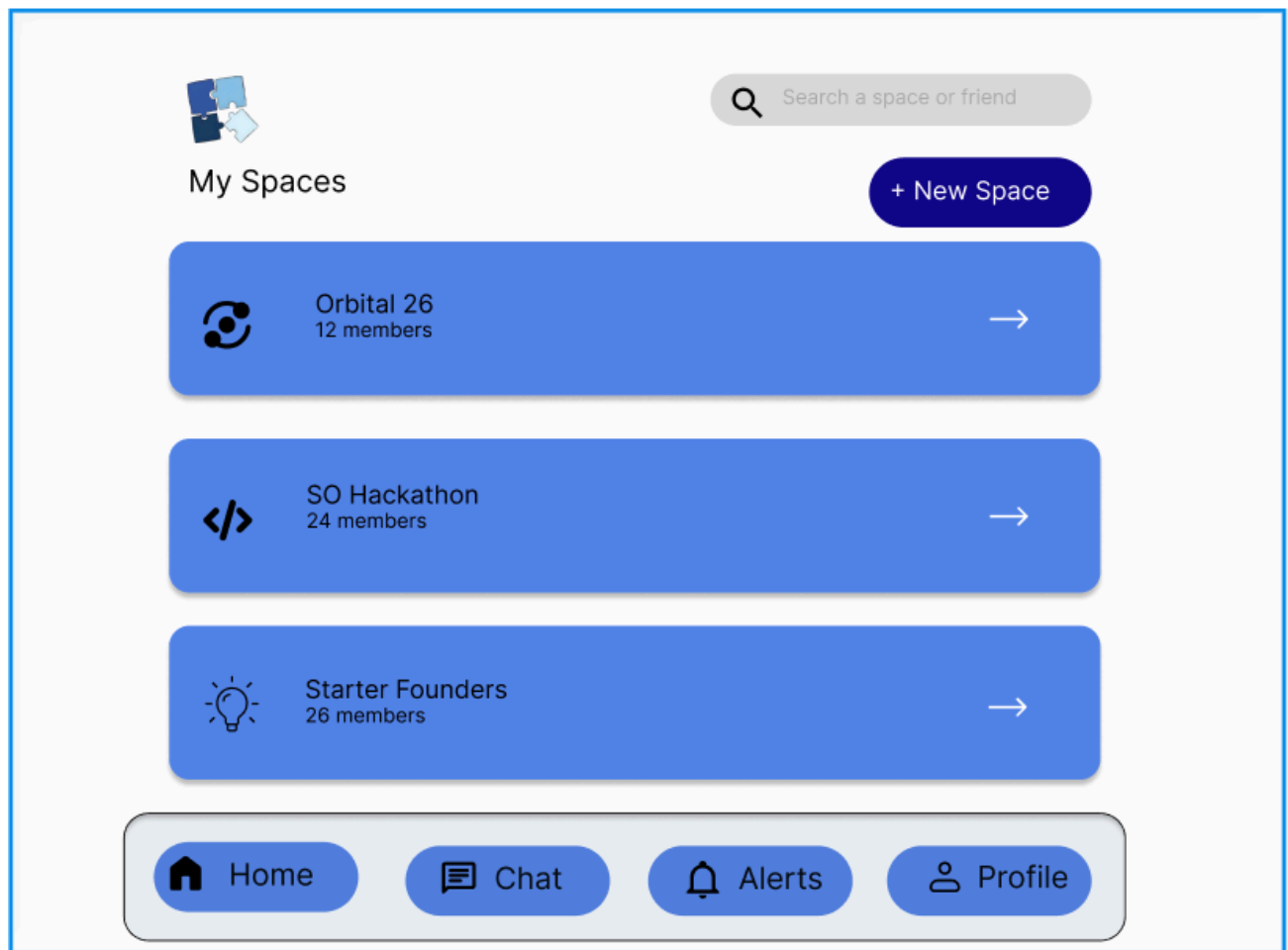
2. Space Editing & Ownership To maintain the accuracy of a Space's information over time while preventing malicious or accidental alterations by random users, we utilize an Ownership model.

- **Permissions:** The user who initially creates the Space is permanently designated as its "Owner." Only the Owner is granted the necessary UI elements and backend permissions to edit the Space's details (Description and Links). Note that the Space Name remains locked after creation to preserve database integrity.
- **Tracking:** To help users gauge how up-to-date a Space's information is, the system tracks a `last_edited_at` timestamp. When a Space is first created, or whenever the Owner successfully updates the details, this timestamp is automatically updated to reflect the exact date and time of the modification.

Activity Diagram



Mockups



< back

Create New Space



Upload space Icon 

Space Name

Description

External Link

Create Now

 Home

 Chat

 Alerts

 Profile

04: Post

Within each dedicated Space, users can create and manage Recruitment Posts. These posts serve as the primary vehicle for team formation, allowing users to broadcast their needs for specific competitions or projects (e.g., a recruitment post for a specific "Orbital 2026" project within the Orbital Space).

1. Post Content & Structure

To ensure clarity and high-quality matching, each post consists of the following data fields:

- Title (Required)
- Description
- Media
- Open Roles & Vacancies (Required)
- Requirements for Becoming a Team Member

2. User Interaction & Engagement

Each post includes a footer to facilitate community engagement:

- Like
- Public Comment
- Apply button

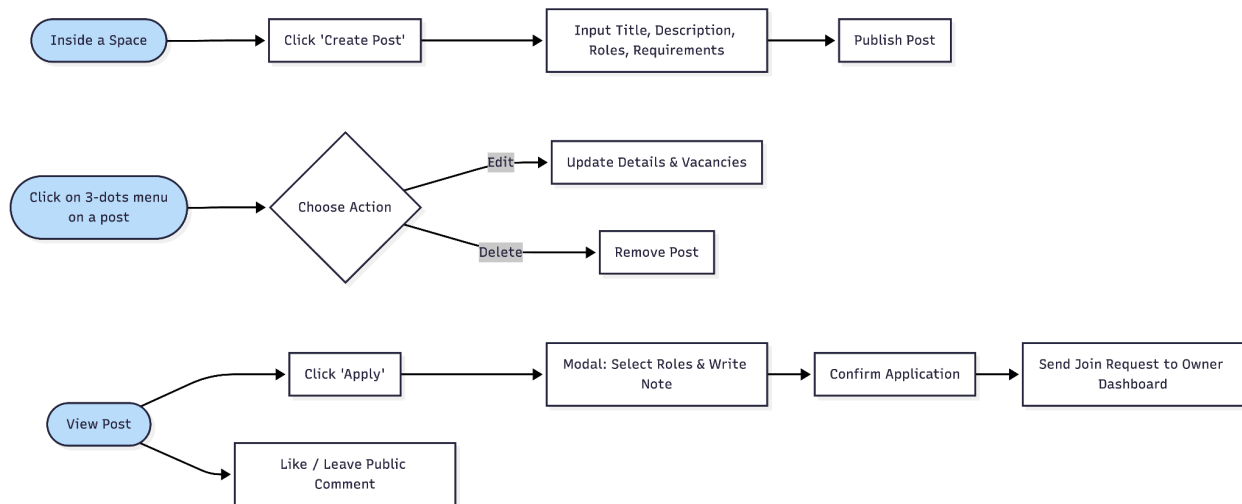
3. Structured Application Workflow

1. Interested users click the "Apply" button on the post footer.
2. A modal (pop-up) window appears, presenting the list of open roles. Applicants can select one or multiple roles that align with their expertise and write a short note to the recruiters.
3. Upon confirmation, a formal join request that contains the selected roles, the message and the applicant's profile link is sent to the post owner's dashboard for review.

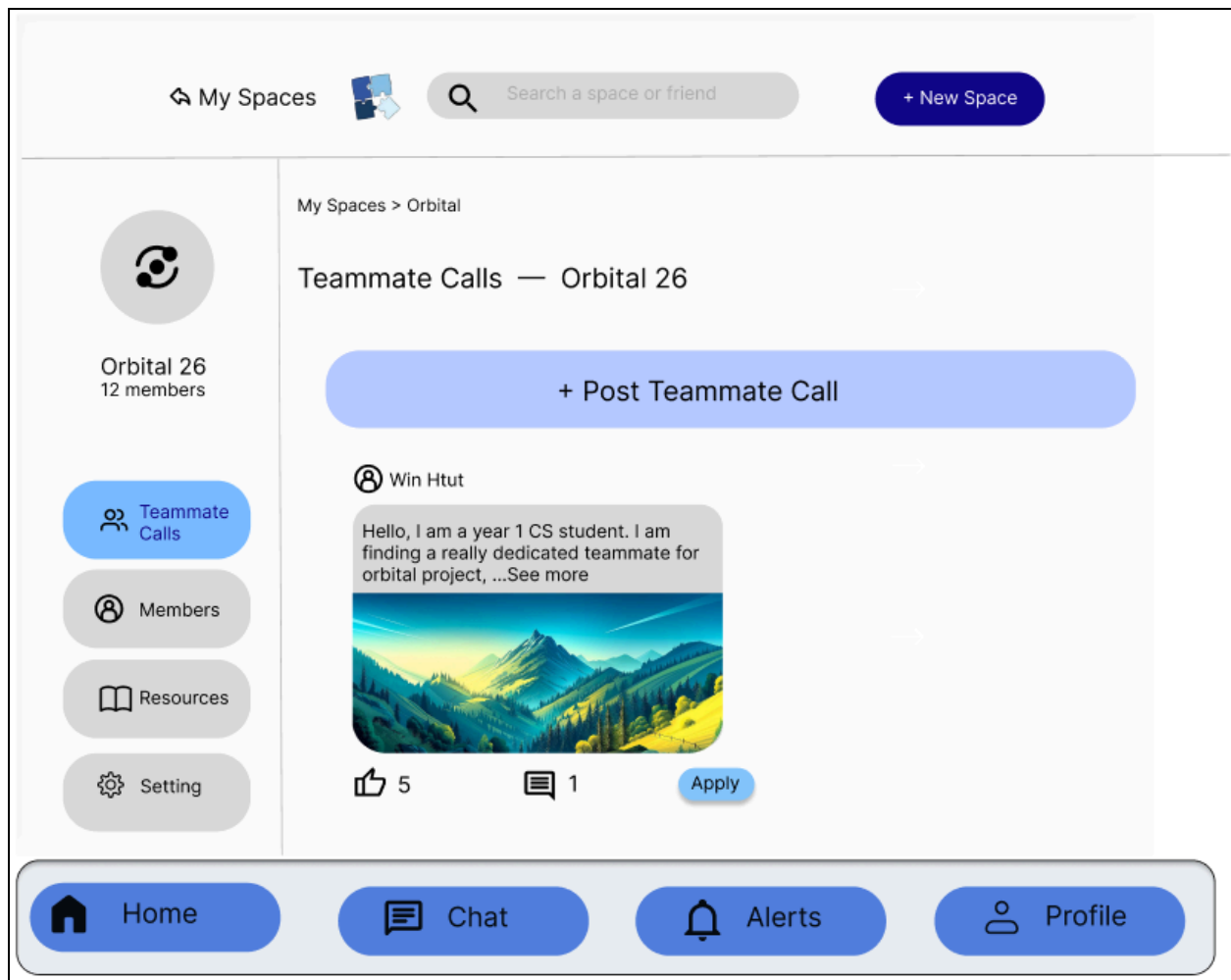
4. Post Management & Governance

1. **Edit:** Owners can update project descriptions, modify requirements, or adjust vacancy counts as the team grows.
2. **Delete:** Owners may remove the post once a team is fully formed or if the project is no longer active.

Activity Diagram



Mockups



 My Spaces 

 Search a space or friend

[+ New Space](#)

[Post Call For Teammate](#)

 Home

 Chat

 Alerts

 Profile

05: Dashboard

Overview

The Dashboard serves as the centralized command center for users to manage their collaborative endeavors on the platform. It provides a comprehensive overview of a user's application lifecycle—tracking both their outbound efforts to join existing teams and the inbound requests from peers applying to their projects. The Dashboard is divided into two primary sections: **My Applications** and **Requests Received**.

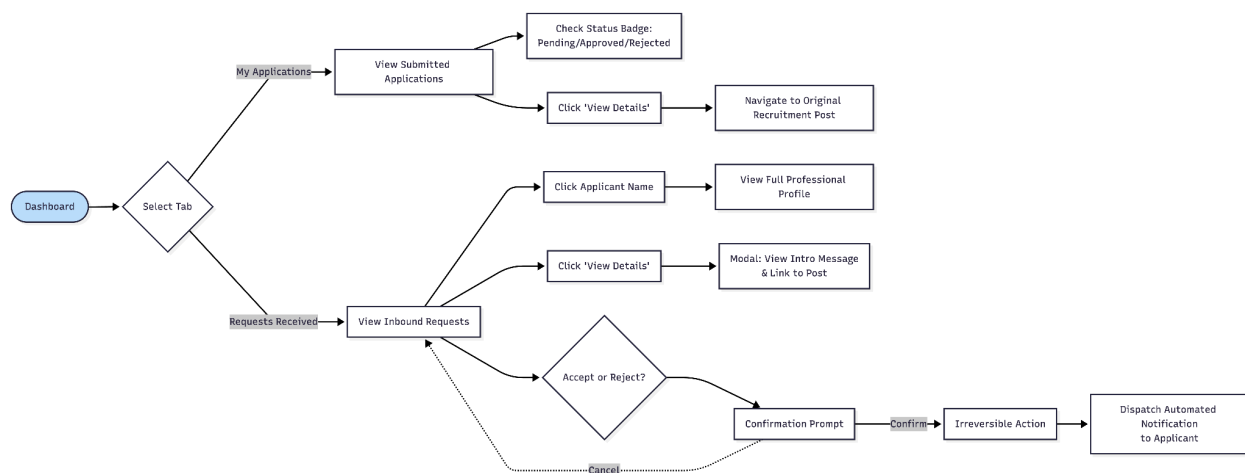
1. My Applications

- Users can view a complete list of their submitted applications.
- Each application features a dynamic status badge clearly indicating the current stage of the process (e.g., Pending, Approved, or Rejected).
- A "View Details" button allows users to seamlessly navigate back to the original recruitment post, helping them recall specific project requirements and details about the team they applied to.

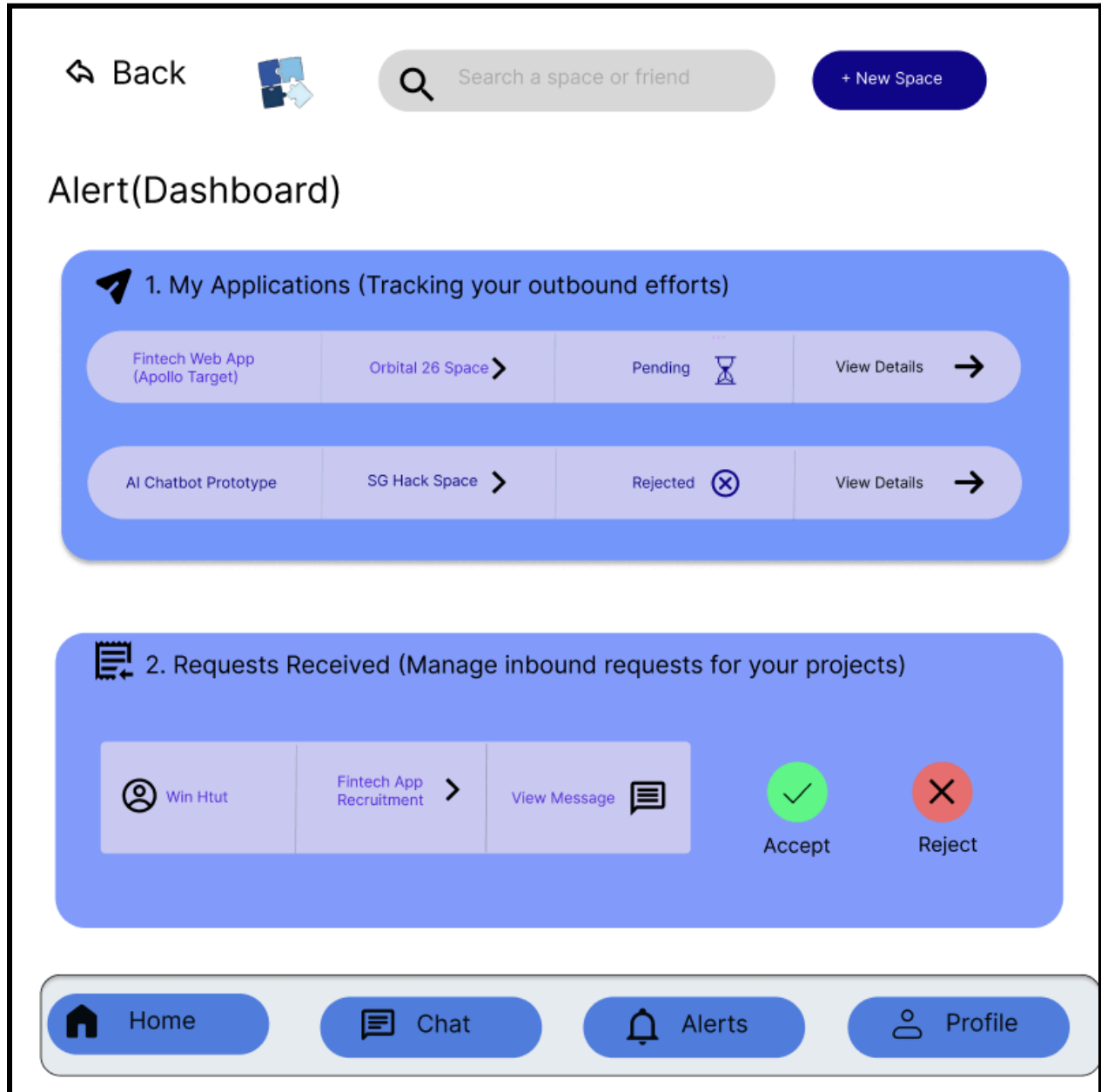
2. Requests Received

- Project owners can click directly on an applicant's name to view their full professional profile, enabling a quick assessment of their skills, experience, and compatibility with the project.
- Clicking "View Details" opens a modal displaying the applicant's personalized introductory message alongside a convenient link back to the user's original recruitment post.
- Each request features prominent "Accept" and "Reject" action buttons.
- To prevent accidental approvals or rejections, clicking either decision button triggers a confirmation prompt. Once confirmed, the action is irreversible, and the system immediately dispatches an automated notification to the applicant regarding the final decision.

Activity Diagram



Mockups

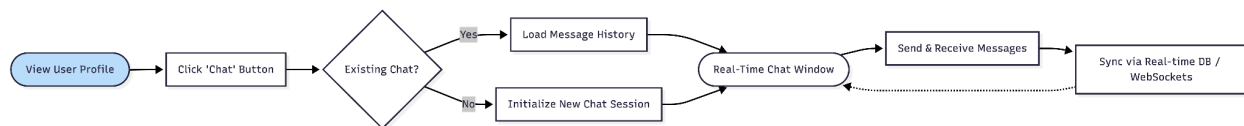


06: Real-time 1-on-1 Chat

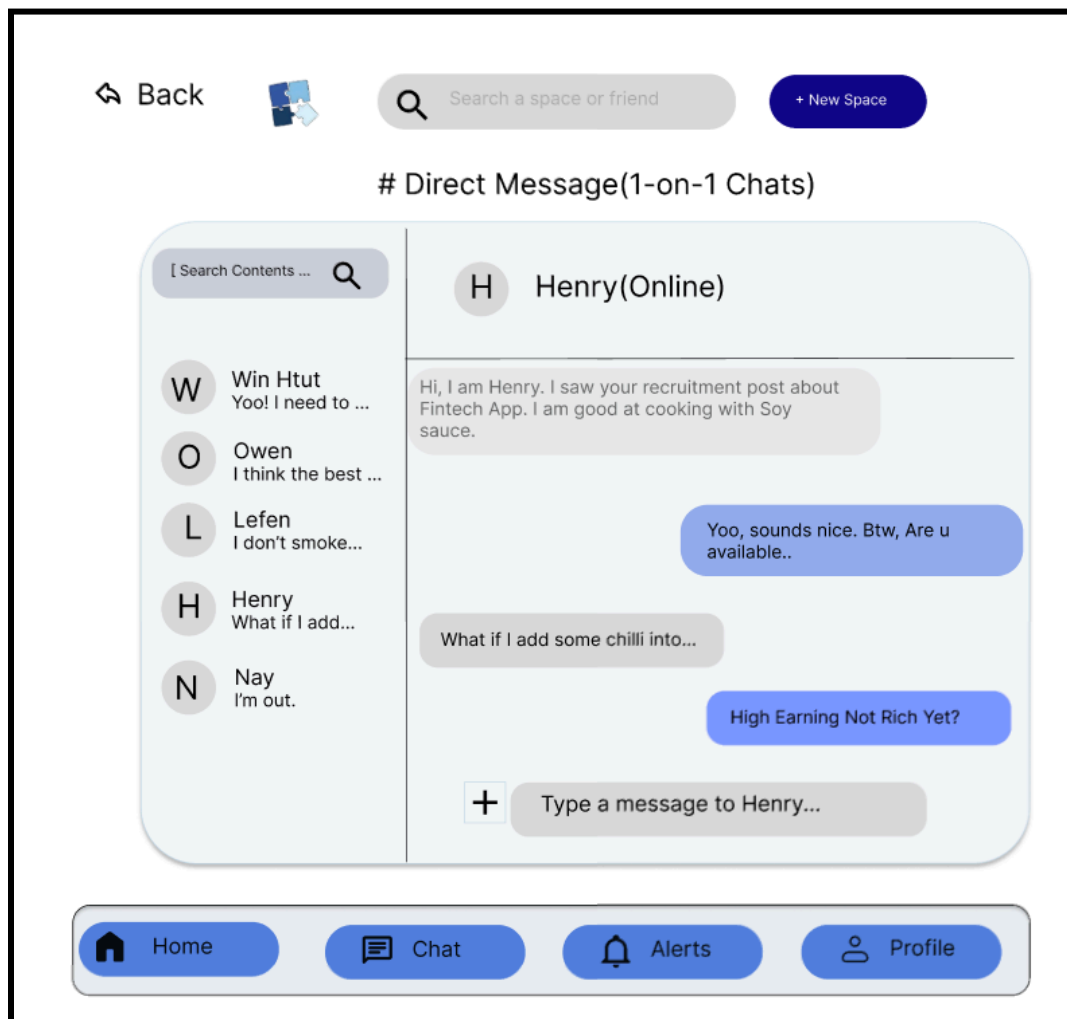
Overview

Finding a teammate requires more than just matching technical skills; it requires assessing communication style, personality, and mutual expectations. The Real-Time 1-on-1 Chat feature bridges this gap, allowing prospective teammates to converse privately before formally committing to a team via the application system. Conversations can be initiated directly by clicking the “chat” button on a user’s profile.

Activity Diagrams



Mockups



Tech Stacks

1. Next.js for backend and frontend
2. Supabase for User authentication and database
3. GitHub for Version Control
4. Vercel for Deployment

Software Engineering Principles

The following principles highlight the core architectural decisions implemented in the current iteration of the platform:

Separation of Concerns (SoC)

To ensure that CoMatch remains maintainable, scalable, and resilient as the codebase expands, we strictly enforce the Separation of Concerns (SoC) principle throughout our architecture. By delineating the application into discrete, minimally overlapping modules, we ensure that each layer addresses a specific functional requirement. Given our modern Next.js and Supabase stack, we executed SoC across the following architectural boundaries:

1. Separation of Client and Server Logic (Next.js Architecture) While Next.js allows blending frontend and backend paradigms, we established a rigorous boundary between the two:

- **UI/Presentation Layer (Client Components):** Components requiring browser APIs or active state management (such as toggling active states in our [NavigationBar.tsx](#)) are strictly declared as client components.
- **Security & Routing Layer (Server Logic):** Crucial logic, such as evaluating user sessions to protect the [/profile](#) route, is entirely abstracted away from the UI components. We enforce this via Next.js [middleware.ts](#), which intercepts requests and redirects unauthenticated users before the client even attempts to render the page.

2. Data Access Abstraction (Supabase Layer) To prevent our React UI components from becoming tightly coupled to database configuration, we abstracted all Supabase initializations into dedicated utility modules.

- Instead of exposing database URLs and configuration keys inside our page files, we structured our connection logic into environment-specific files ([utils/clients.ts](#) for browser-side cookie reading and [utils/server.ts](#) for backend operations).
- Consequently, when the [ProfilePage](#) requires data, it simply invokes a centralized [createClient\(\)](#) function. If our database connection protocols change, we only update the utility file, leaving the UI components completely untouched.

3. Isolation of Authentication and Authorization Authentication (verifying user identity) and Authorization (verifying user permissions) are engineered as distinct, decoupled systems:

- **Authentication:** Managed via [@supabase/ssr](#), which establishes identity by securely storing and parsing encrypted session tokens within server-readable browser cookies.
- **Authorization (Row-Level Security):** Rather than relying on fragile UI logic to determine what a user can edit, authorization is enforced strictly at the database level using PostgreSQL Row-Level Security (RLS). For example, our [profiles](#) table is governed by RLS policies that inherently reject [INSERT](#) or [UPDATE](#) commands unless the request carries a cryptographically verified [authenticated](#) role. This ensures absolute data integrity regardless of frontend behavior.

4. Modular UI Component Design (DRY Principle) We engineered our UI components to be heavily declarative and modular, separating visual presentation from structural layout.

- Rather than hardcoding navigation buttons into multiple pages, we developed a single, independent [NavigationBar](#) component that maps over a centralized [navItems](#) array.
- Furthermore, we injected this component directly into the [RootLayout \(app/layout.tsx\)](#). By isolating the navigation to the root structure, the component automatically persists across all views without requiring redundant imports in individual page files.

Technical Proof of Concept

Integration of Essential Components

To justify the software engineering effort of transitioning from our initial proposal to a functional prototype, a secure and scalable architecture that connects the following technical building blocks has been established.

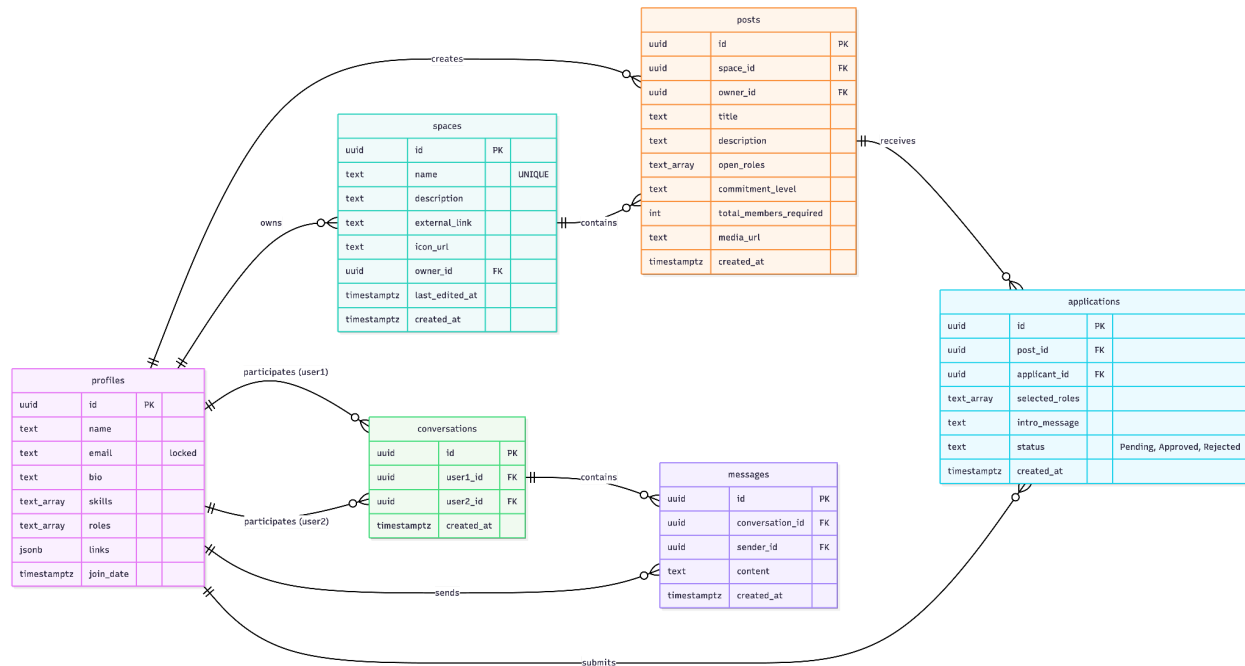
- **The Frontend (Client UI):** Built with Next.js and React, providing the interactive user interfaces (login forms, dashboard layouts, and profile editing fields).
- **The Backend (Server Logic):** Utilizing Next.js Server Components and API routes to securely process user requests and enforce the Separation of Concerns (SoC) outlined in our proposal.
- **The Authentication System:** Integrated with Supabase Auth to securely verify user identities, handle sign-ups/logins, and manage encrypted sessions.
- **The Database:** Configured a Supabase PostgreSQL database, specifically establishing the relational profiles table to persistently store user data.

Implemented Features & System Integration

- **Feature 01: User Authentication (Login & Registration)**

- **The Flow:** A user accesses our Next.js frontend and inputs their credentials. We have fully implemented authentication (Email/ password, Google and LinkedIn Login) as our initial proof of concept.
- **Integration Details:** The Next.js client securely transmits these credentials to Supabase Auth. Upon successful verification, Supabase issues a secure session token. We then utilize Next.js routing to automatically redirect the authenticated user to the home page.
- **Engineering Effort:** This required configuring the Supabase client specifically for the Next.js environment, managing secure session persistence across page reloads, and building robust frontend error handling and validation for user inputs (e.g., catching invalid passwords or unregistered emails).
- **Feature 02: User Profile Building**
 - **The Flow:** Authenticated users can navigate to their personal profile page, which acts as their digital resume. By clicking into "Edit Mode," they can modify public-facing data such as their Bio, Technical Skills, and Active Roles. Core identity metrics (like User ID and Join Date) remain securely locked.
 - **Integration Details:** When the profile page loads, Next.js fetches the user's existing data from the profiles table in our Supabase PostgreSQL database to safely pre-populate the UI forms. When a user saves their updates, the client triggers a secure database mutation. The backend verifies the user's active session before committing the changes to the database.
 - **Engineering Effort:** This involved designing the relational database schema in Supabase, implementing complex React state management to handle the toggling between viewing/editing modes, and ensuring that any changes successfully written to the database are instantly reflected on the UI without requiring a full page refresh.

Database Schema



Project Management

Instruction for Testing

To run the project on your local machine, follow these steps.

1. First, fork the main branch of the [repository](#) to your own GitHub account. Then, clone your fork to your local machine.
2. This project uses pnpm as its package manager. Install the necessary dependencies by running:


```
pnpm install
```
3. Request .env file at [@its_win_here](#) on Telegram.
4. Run the development server by running `pnpm run dev`
5. Open <http://localhost:3000> in your browser.
6. You will be automatically redirected to the login/ register page.
7. Upon successful login/ registration, you will be redirected to the home page.
8. By clicking the “Profile” button on the navigation on the home page, you will be redirected to the Profile page to view your own profile.
9. You can edit your profile information by clicking the “Edit” button.

Development Plan

Accomplishment	Timeline (finish by)
Lift off	18 May
Create Mockups, polished proposal (or README) with detailed explanations of features	20 May
Pick up necessary technologies (next.js, Supabase, etc.)	31 May
Building login, homepage, and user profile page	10 June
Building space, post and dashboard feature	25 June
Update README about the details of the the features developed	29 June
Building one-to-one chat	10 July
Testing	20 July
Polishing UI and deployment	27 July

Project Log

Task	Date	Win (hrs)	Nay (hrs)	Remarks
Project planning meeting	10 May	3	3	Finalize timeline, core features and decide tools to be used.
Lift Off Poster & Video Scripting and creating	15 May	8	8	Drafted Canva poster design and wrote the ~1 min split video script and created video.
Attended MC1: Web App Development(React), Database(Relational Databases)	16 May	4	4	Learned the basics and set up for React, Overview of how databases works.
Liftoff Submission	18 May	-	-	-
Create Github repo and initial setup	20 May	3	3	Initialised repo, linked to vercel and supabase, pushed boilerplate code for nextjs
Write README	29 May	12	12	Wrote this README, created activity diagrams and mockups, wrote details of each features, defined database schema
Implement Log in and register feature	27 May	8	0	

Implement Profile feature	27 May	0	8	
Milestone 1 Submission	1 June	-	-	-